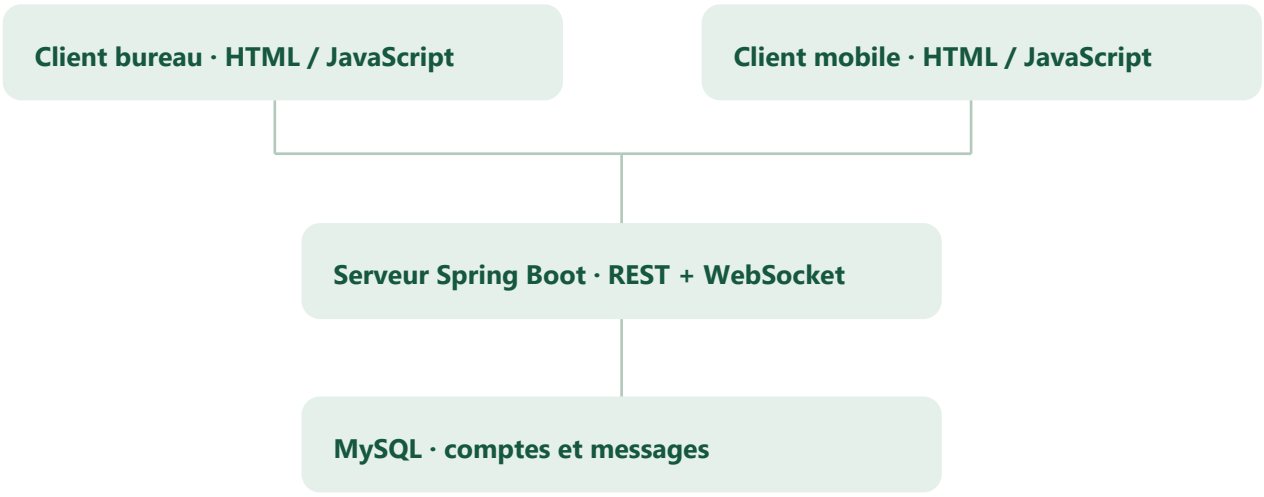
Deux clients, un serveur

Architecture de la messagerie privée · Version de démonstration

Une personne utilise le navigateur de son téléphone, une autre celui de son ordinateur. Chaque client peut créer un compte, choisir un correspondant et reprendre une conversation après une déconnexion.

Architecture déployée



Les deux pages utilisent la même origine que le serveur. Le client bureau juxtapose les contacts et la conversation. Sur mobile, les contacts se parcourent horizontalement au-dessus du fil. Un module JavaScript partagé applique les mêmes règles de connexion et de réception.

Périmètre

Le serveur centralise les échanges privés entre deux comptes. Deux sessions peuvent représenter un même utilisateur. Le modèle prévoit une seule instance Spring Boot, ce qui permet de conserver la présence en mémoire.

Comptes et données

Ce que le serveur conserve entre deux connexions

Utilisateurs

La table users porte un identifiant généré et un pseudo unique. Le serveur dérive une empreinte du mot de passe avec PBKDF2 et un sel. Les réponses adressées aux clients contiennent uniquement les données publiques du compte. La contrainte unique reste valable lorsque deux inscriptions concurrentes choisissent le même pseudo.

Messages

Chaque ligne de messages contient son identifiant, deux références vers users, le contenu et la date attribuée par le serveur. Alice et Bob partagent un historique obtenu en sélectionnant les couples (Alice, Bob) et (Bob, Alice). L'ordre utilise la date puis l'identifiant pour départager deux envois simultanés.

Authentification

POST /api/comptes crée le compte. POST /api/auth/login vérifie le mot de passe et renvoie un JWT signé, valable une heure. Le sujet du jeton identifie l'utilisateur. Le filtre de sécurité valide le jeton Bearer sur les lectures REST. Une requête sans jeton valide reçoit un statut 401.

Le navigateur garde le jeton en mémoire. Le handshake WebSocket transmet le jeton au serveur avant l'ouverture du canal. Le serveur utilise ensuite l'identité de cette session pour enregistrer chaque envoi. Les lectures et écritures sur le canal tiennent compte de l'expiration.

Configuration locale

Le lanceur Windows obtient les identifiants applicatifs depuis le profil de l'utilisateur. La clé de signature peut être configurée avec MESSAGERIE_JWT_SECRET. Par défaut, une nouvelle clé est générée à chaque démarrage et les sessions doivent se reconnecter.

REST et temps réel

Deux échanges réseau aux propriétés différentes

Ouvrir une conversation

GET /api/messages/{utilisateurId} identifie une ressource par une URL. Le serveur combine le correspondant demandé avec l'identité authentifiée. La réponse représente l'historique disponible au moment de la lecture. Le client peut refaire la même requête après une interruption sans créer un message supplémentaire.

HTTP permet de piloter la mise en cache. Un historique privé évolue dès le prochain envoi et dépend du compte connecté. Dans cette version, le client demande une lecture fraîche et Spring Security fournit les en-têtes de protection du cache. Cette lecture ponctuelle reste indépendante du canal temps réel.

Recevoir sans rafraîchir

Après la connexion, le client ouvre /ws/messages. Le serveur peut pousser un message vers son destinataire sur cette connexion bidirectionnelle. Un envoi transporte type, destinataireId et contenu. Le serveur attribue expediteurId, id et dateEnvoi et renvoie le message enregistré aux sessions des deux participants.

Présence et saisie

GET /api/utilisateurs expose le pseudo et le statut de chaque compte. Les clients actualisent cette liste toutes les cinq secondes. La présence dépend des sessions WebSocket ouvertes et valides. Cette actualisation concerne uniquement les contacts ; les messages arrivent directement sur le canal.

Le bonus de saisie utilise un événement typing adressé au correspondant. Le client limite sa fréquence et efface l'indication après un court délai ou à la réception du message. Aucun événement de saisie ne rejoint l'historique.

Déconnexion et reprise

Ordre des opérations, confidentialité et limites

Un message envoyé à un contact hors ligne

Le service valide le destinataire et le contenu, puis enregistre le message dans une transaction MySQL. Le gestionnaire WebSocket tente la livraison après la fin de cette transaction. Si le destinataire a quitté l'application entre ces deux opérations, la ligne enregistrée reste disponible.

À la reconnexion, le client ouvre son canal puis récupère la conversation par REST. Il fusionne les données par identifiant. Si le même message arrive dans la réponse HTTP et sur le WebSocket, une seule bulle apparaît. Un écho de sauvegarde au client expéditeur confirme l'enregistrement, sans garantir la lecture par le destinataire.

Difficultés rencontrées

Les classes historiques nommées dto entraient en conflit avec les packages prévus. Les objets de requête et de réponse ont été regroupés sous payload. Sur Windows, le premier lancement nécessitait un compte MySQL utilisable par Spring. Un lanceur configure la base et conserve les identifiants dans le profil local.

Conditions de déploiement

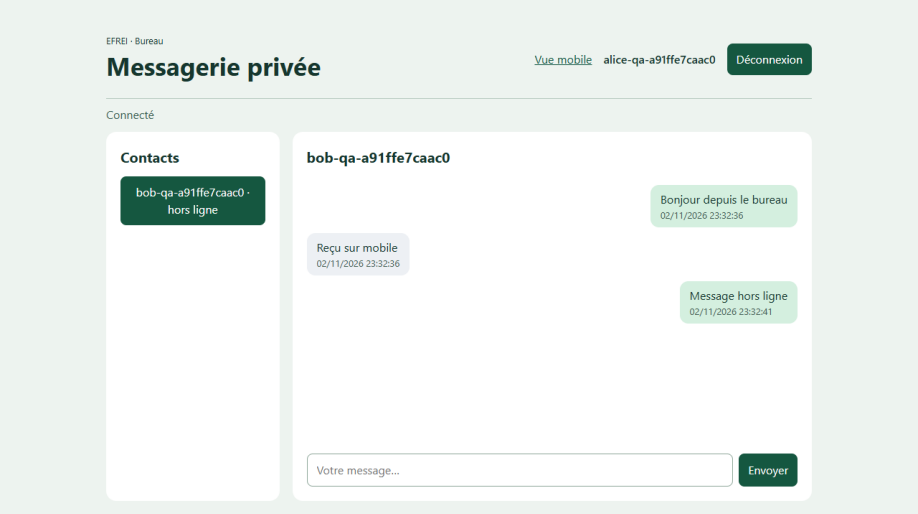
Le test local utilise HTTP. Pour une utilisation sur un réseau non fiable, il faut servir les clients en HTTPS et utiliser WSS. Le paramètre du handshake contient un jeton temporaire et doit être exclu des journaux d'accès. Les mots de passe et les clés de signature restent hors du dépôt.

Le prototype charge la conversation entière. Un déploiement avec des historiques longs demanderait une pagination. Plusieurs instances du serveur devraient aussi partager les événements temps réel et les informations de présence.

Démonstration et vérifications

Résultats locaux et preuve à compléter sur le matériel de soutenance

Deux sessions Edge isolées ont utilisé les clients bureau et mobile sur le même PC. Le test a créé deux comptes, vérifié la présence et l'indicateur de saisie, envoyé un message dans chaque sens, puis enregistré un troisième message pendant la déconnexion du mobile. Après reconnexion, le mobile a retrouvé exactement trois messages.





Observations à reproduire sur les deux appareils

Sur le réseau privé, ouvrir le client mobile avec l'adresse IPv4 du PC et laisser le client bureau ouvert. Utiliser deux comptes distincts. Vérifier la livraison immédiate dans chaque sens, déconnecter le téléphone, envoyer depuis le PC, puis reconnecter le téléphone et retrouver le message.

Les images ci-dessus proviennent du test local. La photo de la démonstration sur deux appareils physiques sera ajoutée après la manipulation. Le README donne les commandes de lancement et de test.